

Definition: Lean Inductive Types for the Universal Graph-Token DSL

Lean Inductive Types for the Universal Graph-Token DSL

This page defines the **canonical Lean type system** corresponding to the universal math graph-token grammar. These inductive types form the foundation of the DSL layer (Layer 1) and provide a typed, reversible structure that maps directly to the IR (Layer 2). All entities, relations, operations, and polarity rules are encoded at the type level.

This specification is stable and intended for encoder/decoder implementation and SLM training.

1. Polarity (Balanced Ternary)

Polarity encodes constructive, neutral, and reductive mathematical steps.

```
inductive Polarity
| pos -- constructive (+)
| zero -- neutral (0)
| neg -- reductive (-)
```

2. Entity Identifiers

Entity identifiers represent mathematical objects across all domains.

```
inductive EntityId
| var : String → EntityId
| num : Int → EntityId
| term : Nat → EntityId
| eqn : EntityId
| fn : Nat → EntityId
| mat : Nat → EntityId
| vec : Nat → EntityId
| set : Nat → EntityId
| rv : Nat → EntityId
| op : Nat → EntityId
| obj : Nat → EntityId
| mor : Nat → EntityId
| custom : String → EntityId
```

3. Entity Structure

Matches IR tokens like E0:X*.

```
structure Entity :=  
  (class : String := "E")  
  (polarity : Polarity)  
  (id : EntityId)  
  (chain : String := "*")
```

4. Attributes

Attributes attach metadata to entities.

```
structure Attribute :=  
  (class : String := "A")  
  (polarity : Polarity)  
  (target : EntityId)  
  (key : String)  
  (value : String)  
  (chain : String := "*")
```

5. Relation Operators

Universal relation vocabulary across all mathematical domains.

```
inductive RelationOp  
  | mul | add | sub | div | eq  
  | diff | integrate | limit | eval_at  
  | matmul | dot | cross | transpose | inv | det  
  | union | intersect | set_sub | subset  
  | and | or | not | implies | iff  
  | prob | expect | var | dist  
  | compose | id | assoc  
  | custom : String → RelationOp
```

6. Relation Structure

Matches IR tokens like:

```
R+:mul*:E0:X*:E0:N3*:E0:T1*
```

```
structure Relation :=  
  (class : String := "R")  
  (polarity : Polarity)  
  (op : RelationOp)  
  (e1 : Entity)  
  (e2 : Entity)  
  (e3 : Entity)  
  (chain : String := "*")
```

7. Operation Operators

Rewrite steps across all mathematical domains.

```
inductive OperationOp  
  | add | sub | div | iso  
  | apply_diff_rule | apply_int_rule | change_var | limit_simplify  
  | row_reduce | diag | eig_decomp  
  | simplify_logic | simplify_set | normalize_formula  
  | custom : String → OperationOp
```

8. Operation Structure

Matches IR tokens like:

```
O-:sub*:E0:N5*:E0:EQ*  
structure Operation :=  
  (class : String := "O")  
  (polarity : Polarity)  
  (op : OperationOp)  
  (e1 : Entity)  
  (e2 : Entity)  
  (chain : String := "*")
```

9. Graph Structure (Full DSL Object)

Represents an entire graph-token block.

```
structure Graph :=  
  (entities : List Entity)  
  (attributes : List Attribute)  
  (relations : List Relation)  
  (operations : List Operation)
```

Purpose of This Page

This page defines the **Lean DSL type layer** that corresponds exactly to the universal graph-token IR. It is the foundation for:

- IR encoder (Lean $\rightarrow$ tokens)
- IR decoder (tokens $\rightarrow$ Lean)
- SLM rewrite engine
- VS Code autocompletion integration

This type system is canonical, reversible, and ready for implementation.